

RELATÓRIO TÉCNICO DE ARQUITETURA DE SOFTWARE

Framework Extensível para Jogos de Quiz em Java (POO / PPP)

EQUIPE DO PROJETO / DISCENTES

Paulo Fernando A. de Carvalho
Matrícula: 12421BCC056

Cauê Fernando Abre
Matrícula: 12421BCC064

João Pedro de M. e Menezes
Matrícula: 12421BCC087

1. VISÃO GERAL DO SISTEMA

O sistema consiste em um **Framework Genérico e Reutilizável para Jogos de Quiz**, desenvolvido sob os preceitos do paradigma de Orientação a Objetos. A solução foi projetada com foco em desacoplamento, separando estritamente o mecanismo central de execução da aplicação (*core framework*) das implementações de domínio e interfaces de usuário (*aplicação concreta*).

2. PADRÕES DE PROJETO UTILIZADOS (DESIGN PATTERNS)

- Template Method (Camada Engine):** Implementado na classe abstrata `QuizTemplate`. Define o algoritmo/esqueleto fixo de execução do jogo (início, iteração de perguntas, captura de resposta, cálculo de pontos e término), delegando para as subclasses apenas a busca dos dados e a forma de exibição.
- Strategy (Camada Scoring):** Interface `PontuacaoStrategy` e suas implementações concretas (`PontuacaoFixaStrategy` e `PontuacaoComPenalidadeStrategy`). Permite alternar dinamicamente a regra do cálculo da pontuação sem alterar a lógica de controle da partida.
- Factory Method (Camada Factory):** A classe `PerguntaFactory` centraliza a instanciação das perguntas, garantindo o encapsulamento do processo de criação dos objetos do modelo.

3. ESTRUTURA DE PACOTES E MÓDULOS

Pacote	Componentes Principais	Responsabilidade / Descrição
<code>quiz.framework.engine</code>	<code>QuizTemplate</code>	Fluxo principal do motor do quiz (Template Method).
<code>quiz.framework.model</code>	<code>Pergunta</code> , <code>ResultadoQuiz</code>	Entidades de dados do domínio e estatísticas do jogo.
<code>quiz.framework.scoring</code>	<code>PontuacaoStrategy</code> , Concrete Strategies	Algoritmos de pontuação e penalidade.
<code>quiz.framework.view</code>	<code>QuizView</code>	Interface abstrata para desacoplamento de UI.
<code>quiz.aplicacao.*</code>	<code>QuizMatematica</code> , <code>QuizProgramacao</code> , <code>ConsoleQuizView</code> , <code>SwingQuizView</code>	Implementações concretas dos jogos em CLI (Console) e GUI (Java Swing).

4. CONCLUSÃO

A arquitetura atingiu alto nível de coesão e baixo acoplamento. Através do uso de interfaces e classes abstratas, é possível adicionar novos modos de jogo, regras de pontuação ou interfaces visuais (web/mobile) sem a necessidade de alterar o código do motor principal (`quiz.framework`).